Conference Paper Title*

Josè Dalla Torre
Machine Learning and Perception Lab
University of Udine
Udine, Italy
dallatorre.josezito@spes.uniud.it

Christian Micheloni
Machine Learning and Perception Lab
University of Udine
Udine, Italy
christian.micheloni@uniud.it

Gian Luca Foresti
Machine Learning and Perception Lab
University of Udine
Udine, Italy
gianluca.foresti@uniud.it

***Abstract*—Write this last, once your experiments are complete. It should contain five elements: 1. Context: Ball action spotting aims to identify and temporally localize ball-related events in broadcast soccer videos. 2. Problem: Accurate localization is challenging because events are brief and densely distributed. 3. Method: You investigate replacing the 3D CNN temporal encoder with a Transformer while retaining the 2D visual backbone. 4. Experiments: Describe the dataset, evaluation metric, and comparison. 5. Results and conclusion: Report your actual results and the main finding. Aim for approximately 150–200 words, depending on the conference requirements.**

***Index Terms*—Ball action spotting, soccer video analysis, temporal action localization, 3D convolutional neural networks, Transformers, video understanding.**

I. Introduction

Paragraph 1 — General problem Introduce video understanding and temporal action spotting.

Explain that action spotting involves identifying both the semantic category and the temporal location of events in long, untrimmed videos.

Then introduce soccer as an interesting application because broadcast videos contain many events, and understanding them requires temporal reasoning.

The improvements in deep learning and increase computing power permitted considerable progress in the field of video understanding. In particular sport video understanding field has become more important area for computer vision researcher [1] [2] [3] moving from easier task like object detection to more demanding challenges like temporal action spotting.

Temporal action spotting is a computer vision task that identifies the exact timestamp when a specific action or event begins or occurs in an untrimmed video, without needing to outline its full start-and-end duration.

Paragraph 2 — Ball action spotting

To advance research in soccer video understanding the SoccerNet Challenge was proposed at CVPR 2023. The challenge focused on 7 tasks divided into three categories: Broadcast video understanding, Field Understanding, and Player understanding. We will focus on the first edition of Ball action spotting challenge included in the first category.

A. Cioppa et al. [4] define ball action spotting challenge as localizing the exact time when two types of actions related to the soccer ball occur: pass and drive. This task pose a unique challenge due to the dynamic and complex nature of the game. During a match players move rapidly, often obscuring each other, making it difficult to accurately track their movements. In this challenge a precise timestamp is set for this actions:

- **pass event**: the timestamp correspond to the moment when the ball leaves a player's foot
- **drive event**: the timestamp corrrespond to the moment when a player control the ball

The dataset consist of 7 games with 11,041 annotated timestamps with 2 games not publicy available for the challenge set. The recordings of the games are untrimmed and provided by Footovision. Other details of the challenge can be found at SoccerNet Ball Action Spotting challenge.

Narrow the problem to ball-related events.

Explain: - What ball action spotting is. - The difference between general soccer action spotting and ball action spotting. - Why precise temporal localization matters. - Why ball actions can be difficult to distinguish. Mention the relevant SoccerNet challenge and dataset.

Paragraph 3 — Existing approaches and research gap Introduce the main approaches.

For example: - CNN-based approaches extract visual information from individual frames. - 3D convolutions can learn spatiotemporal features from neighboring frames. - Transformers use self-attention to model relationships between temporal representations. Explain that your work investigates whether self-attention provides a useful alternative to convolutional temporal modeling.

Be careful: don't claim that existing CNN approaches are inadequate. Your experiments should establish whether the alternative is beneficial. Paragraph 4 — Your approach Describe your idea briefly: - Start from the architecture of the 2023 winning solution. - Retain the shared 2D visual feature extractor. - Replace the 3D CNN temporal encoder with a Transformer encoder. - Compare the resulting architecture with the convolutional baseline. Contributions End the Introduction with a short contribution list.

The main contributions of this work are:

- We investigate a Transformer-based temporal encoder for ball action spotting, replacing the 3D convolutional component of an existing architecture.

- We describe an architecture that combines a shared 2D visual feature extractor with self-attention-based temporal feature aggregation.
- TODO: check this part We experimentally compare convolutional and Transformer-based temporal modeling in terms of spotting performance and computational efficiency.

In order to understand the salient actions of a broadcast soccer game, SoccerNet introduces the task of action spotting, which consists in finding all the actions occurring in the videos. Beyond soccer understanding, this task addresses the more general problem of retrieving moments with a specific semantic meaning in long untrimmed videos.
Introduce temporal action spotting in broadcast soccer. Explain that the model must identify both the action category and its timestamp in an untrimmed video, and that ball events can be brief and closely spaced. Introduce temporal 3D convolutions as a way to learn features across neighboring frames. Explain their local temporal processing and why you want to investigate self-attention as an alternative. Avoid claiming that convolution is inherently inadequate; your experiments should determine whether the alternative helps. Explain that your work investigates whether changing the temporal feature encoder, while preserving the rest of the pipeline, changes accuracy and efficiency on this particular task. The gap should be supported by related literature, not merely by the fact that you implemented a Transformer. List only contributions you actually make. For example:

- An implementation of a Transformer-based temporal encoder in an existing ball action spotting pipeline.
- A controlled comparison against a 3D CNN temporal encoder.
- An analysis of accuracy, temporal context, and computational cost.

II. Related Work

A. Soccer Action Spotting

Soccer action spotting has received increasing attention in computer vision, particularly with the introduction of the SoccerNet benchmark [3]. The benchmark provides broadcast soccer videos annotated with temporally localized events, enabling the development and evaluation of methods for detecting relevant moments in long, untrimmed recordings.

The SoccerNet dataset and its subsequent extensions have supported research into different aspects of soccer video understanding, including action spotting and the recognition of events from broadcast footage [1], [4]. These tasks require models to process long sequences and identify events whose temporal boundaries or occurrence times are not known in advance.

The SoccerNet 2023 challenges further expanded this research direction by introducing several tasks addressing broadcast video, field, and player understanding [4]. Within this framework, Ball Action Spotting focuses specifically on identifying two ball-related events: passes and drives. Each event is represented by a timestamp corresponding to a precisely defined interaction with the ball. This formulation differs from broader action spotting tasks because the target events are closely related to changes in ball possession and control.

The temporal precision required by this task makes the extraction of information from neighboring frames particularly important. A single frame may not provide sufficient context to distinguish the relevant interaction, motivating architectures that combine spatial feature extraction with temporal modeling.

B. Ball Action Spotting

The SoccerNet Ball Action Spotting Challenge 2023 attracted several approaches based on deep learning architectures and different training strategies. Among them, Baikulov achieved first place with an efficient architecture combining 2D and 3D convolutional operations [6].

The proposed solution adopts a slow-fusion strategy. A shared 2D convolutional backbone independently extracts visual features from groups of neighboring grayscale frames. These feature representations are subsequently processed by a 3D convolutional encoder, which captures spatiotemporal information. Generalized mean pooling is then applied to aggregate the resulting features before the final classification stage.

An important aspect of this approach is the separation of spatial and temporal feature extraction. By applying 2D convolutions before 3D convolutions, the architecture first extracts visual information from individual frame groups and subsequently integrates information across time. This design provides an effective balance between spatial representation learning and temporal feature extraction.

Beyond the model architecture, Baikulov's solution employs a multi-stage training strategy. This includes pretraining on the SoccerNet Action Spotting dataset, transfer learning, and fine-tuning with longer temporal sequences. The solution also introduces an out-of-fold prediction-based sampling strategy to increase the probability of selecting challenging training examples. These choices highlight the importance of the training procedure, in addition to the architecture itself, for achieving strong spotting performance.

Another notable approach was proposed by Wang et al. [7], who obtained second place in the 2023 challenge with Boosted Model Ensembling (BME). Their method builds upon the E2E-Spot architecture [5] and focuses on generating and combining multiple candidate models. The candidate models differ in their feature extraction architectures and training configurations. An iterative selection procedure is then used to identify suitable ensemble members and assign their respective weights according to their validation performance.

These two solutions illustrate different strategies for improving ball action spotting: the use of an efficient spatiotemporal architecture and the combination of diverse models through ensemble learning. In particular, Baikulov's slow-fusion architecture provides the reference for the approach investigated in this work. We explore an alternative temporal modeling

strategy by replacing its 3D convolutional encoder with a Transformer-based encoder while retaining the shared 2D visual feature extractor.

C. Convolutional Temporal Modeling

Convolutional neural networks are widely used in video understanding because they can learn hierarchical visual representations directly from image data. A conventional 2D CNN applies convolutional filters across the spatial dimensions of an image, extracting features such as edges, textures, and higher-level semantic patterns.

However, video data also contains temporal information that cannot be fully represented by processing frames independently. Three-dimensional convolutional neural networks address this limitation by extending convolutional operations to the temporal dimension. Their filters operate across both spatial and temporal dimensions, allowing the network to learn local motion patterns and interactions between neighboring frames.

An alternative to applying 3D convolutions throughout the entire network is the slow-fusion strategy. In this approach, spatial features are initially extracted using 2D convolutions and are progressively integrated through subsequent temporal processing stages. This reduces the need to perform full spatiotemporal convolutions at the earliest network layers while retaining the ability to learn temporal representations.

The slow-fusion architecture employed by Baikulov [6] follows this principle by combining a shared 2D feature extractor with a 3D convolutional temporal encoder. The resulting architecture serves as the convolutional reference for our investigation.

D. Transformer-Based Temporal Modeling

Transformers were originally introduced as an attention-based architecture for sequence modeling. Their central component is the self-attention mechanism, which enables each element of a sequence to incorporate information from other elements through interactions between query, key, and value representations.

Unlike convolutional operations, which aggregate information through locally defined receptive fields, self-attention computes relationships between elements across the input sequence. This allows the model to capture dependencies between temporally separated representations without requiring information to pass exclusively through successive local operations.

Since Transformers do not inherently encode the ordering of input elements, positional information is incorporated into the sequence representations. Positional embeddings provide the model with information about the relative or absolute positions of individual elements, allowing temporal order to influence the attention computation.

In video understanding, this mechanism can be applied to sequences of visual feature representations extracted from individual frames or groups of frames. A temporal encoder can then learn relationships between these representations and aggregate the resulting information for video-level classification.

In this work, we investigate this approach specifically for ball action spotting. Rather than processing spatial feature maps through a 3D convolutional encoder, the proposed architecture first aggregates spatial information into compact feature vectors and subsequently models their temporal relationships using a Transformer encoder. This provides an alternative to the convolutional temporal modeling strategy employed by the reference architecture.

III. PROPOSED METHOD

A. Architecture Overview

The proposed model is a single-stage architecture designed to detect ball-related actions by jointly exploiting spatial appearance and temporal information. It follows a two-part feature extraction strategy: a convolutional backbone extracts visual representations from short groups of consecutive frames, while a factorized Transformer models temporal dependencies and spatial relationships within the resulting feature maps.

Given an input video segment containing $T = 15$ grayscale frames, the frames are divided into five non-overlapping groups of three consecutive frames. Each group is processed independently by a shared 2D convolutional backbone. The resulting feature maps are projected to a common channel dimension and arranged as a spatio-temporal feature tensor.

Unlike approaches that aggregate spatial information before temporal modelling, the proposed architecture preserves a coarse spatial representation throughout the Transformer stages. Temporal attention is first applied independently at each spatial location. Subsequently, spatial attention enables information exchange between different locations within each time step. This factorized design allows the network to model both the temporal evolution of visual features and their spatial relationships.

The final representation is obtained by averaging the encoded features over the temporal and spatial dimensions. A linear classification layer then produces the predictions for the two ball-action categories considered in the 2023 challenge: pass and drive.

B. Frame Grouping and 2D Feature Extraction

Let the input video segment be represented by

$$X \in \mathbb{R}^{B \times T \times H \times W},$$

where B is the batch size, $T = 15$ is the number of input frames, and H and W denote the frame height and width.

To exploit short-term visual information while maintaining the computational efficiency of a 2D backbone, consecutive frames are grouped into stacks of three. The input is therefore reshaped into five groups:

$$X_s \in \mathbb{R}^{(B \cdot 5) \times 3 \times H \times W}.$$

Each group is treated as a three-channel input and processed by the same 2D convolutional backbone. The backbone extracts a feature map for each group, producing a representation with 192 channels and a spatial resolution of 23×40 in the configuration shown in the implementation.

A 1×1 convolution followed by batch normalization and a SiLU activation is used to project the extracted features to a 192-dimensional representation. The projected feature maps are then reshaped to recover the temporal dimension:

$$F \in \mathbb{R}^{B \times 5 \times 192 \times 23 \times 40}.$$

At this stage, each of the five temporal positions is represented by a spatial feature map. The shared convolutional backbone provides visual descriptors, while the subsequent Transformer stages explicitly model their spatio-temporal relationships.

C. Spatial Feature Reduction

Directly applying attention to the full spatial resolution would result in a large number of tokens and substantial computational cost. To make attention-based processing more efficient while retaining spatial information, adaptive average pooling is applied independently to each feature map.

The spatial dimensions are reduced from 23×40 to 6×10:

$$F_p \in \mathbb{R}^{B \times 5 \times 192 \times 6 \times 10}.$$

The resulting grid contains 60 spatial locations per time step. Each location is represented by a 192-dimensional feature vector. Importantly, this operation does not collapse the entire feature map into a single vector; it preserves a coarse spatial layout that is subsequently used by the attention modules.

D. Factorized Spatial Temporal Transformer

The central component of the proposed architecture is a factorized Transformer encoder composed of two sequential attention stages: temporal attention and spatial attention.

Temporal attention. First, the feature tensor is reorganized so that each spatial location is associated with a sequence of five feature vectors, one for each temporal group. The tensor is reshaped as

$$F_t \in \mathbb{R}^{(B \cdot 60) \times 5 \times 192}.$$

Consequently, each spatial location is processed as an independent temporal sequence. This design allows the model to learn how the visual representation at a particular location changes across the input segment, without initially mixing information from different spatial locations.

A learnable temporal positional embedding is added to each sequence to encode the ordering of the five temporal positions. The resulting representation is processed by a Transformer encoder consisting of two layers. Each layer uses multi-head self-attention with eight attention heads, a feed-forward dimension of 768, and a dropout probability of 0.1.

The temporal encoder therefore models dependencies between the five frame groups independently at each of the 60 spatial locations.

Spatial attention. After temporal encoding, the tensor is rearranged to restore the temporal and spatial dimensions:

$$F'_t \in \mathbb{R}^{B \times 5 \times 60 \times 192}.$$

Spatial attention is then applied independently to each temporal position. For each of the five time steps, the 60 spatial feature vectors form a sequence:

$$F_s \in \mathbb{R}^{(B \cdot 5) \times 60 \times 192}.$$

A learnable spatial positional embedding is added to represent the locations within the 6×10 feature grid. The resulting sequence is processed by a Transformer encoder with one layer, eight attention heads, a feed-forward dimension of 768, and a dropout probability of 0.1.

This second stage enables the model to exchange information between different spatial locations. In combination with the preceding temporal encoder, it allows the network to represent both how visual features evolve over time and how different regions of the scene relate to one another.

E. Feature Aggregation and Classification

Following the spatial Transformer, the encoded representation is reshaped to

$$F_{\text{out}} \in \mathbb{R}^{B \times 5 \times 60 \times 192}.$$

The temporal and spatial dimensions are averaged to obtain one feature vector per input segment:

$$z_b = \frac{1}{5 \cdot 60} \sum_{t=1}^{5} \sum_{s=1}^{60} F_{\text{out}}[b, t, s, :].$$

The resulting representation has 192 dimensions and summarizes the information extracted across all temporal groups and spatial locations.

Finally, a fully connected layer maps this representation to the output classes:

$$\hat{y}_b = W z_b + b,$$

where the output dimension corresponds to the two ball-action categories, pass and drive. The resulting scores are used to predict the presence of the corresponding actions in the input segment.

—

IV. EXPERIMENTAL SETUP

A. Dataset and Task

The experiments are conducted in the context of the SoccerNet Ball Action Spotting Challenge 2023. The task consists of identifying two types of ball-related actions: passes and drives. A pass corresponds to the moment when the ball leaves

a player's foot, whereas a drive corresponds to an instance in which a player controls the ball.

The dataset comprises broadcast football videos annotated with temporal action labels. The objective is to predict the temporal locations of the annotated actions from the video frames.

[TO BE COMPLETED: Specify the exact number of games and annotations used in your experiments, the training and validation split, and whether the official challenge test set was used for evaluation.]

B. Input Representation

The proposed model receives a sequence of 15 grayscale frames as input. Consecutive frames are grouped into stacks of three, resulting in five temporal groups. Each group is processed by the shared 2D convolutional backbone, and the resulting feature maps are passed to the factorized spatial–temporal Transformer.

[TO BE COMPLETED: Specify the frame sampling rate, input resolution, temporal distance between sampled frames, and any image preprocessing or normalization applied.]

C. Training Configuration

The proposed model was trained following the training procedure of the original Ball Action Spotting Challenge solution by Baikulov [6]. The original training configuration was retained, including the optimizer, learning-rate schedule, number of epochs, and loss function.

The only modification concerned the batch size, which was reduced from 2 to 1 due to GPU memory constraints. While the original implementation was trained on an NVIDIA GeForce RTX 3090 Ti, our experiments were conducted on an NVIDIA GeForce RTX 3080. This adjustment allowed the model to be trained within the available hardware resources while preserving the remaining training settings.

The experimental evaluation considers the model obtained directly after the training procedure. No additional fine-tuning stages, training phases, or post-processing operations were applied to the predictions.

The model was trained using the configuration adapted from Baikulov's reference implementation [6]. The input videos were represented by sequences of 15 grayscale frames, sampled with a temporal step of two frames and grouped into five stacks of three consecutive frames. Each frame was resized and padded to a resolution of 1280 × 736.

The model was optimized using AdamW, with a base learning rate of 3×10^{-4} and a batch size of 1. Training employed mixed-precision computation and an epoch size of 6,000 samples. The training schedule consisted of two stages: a six-epoch warm-up stage followed by a 30-epoch training stage. Focal loss was used, with a focusing parameter $\gamma = 1.2$.

An exponential moving average of the model parameters was maintained with a decay factor of 0.999. The EfficientNetV2-B0 backbone was initialized with pretrained ImageNet-1K weights and was not frozen during training. The batch size was reduced from the reference configuration to accommodate the available GPU memory.

D. Dataset Partitioning and Training Protocol

We adopted the seven-fold cross-validation strategy of Baikulov [6]. Each labelled game was assigned to one fold. For each fold, six folds were used for training, while the remaining fold was reserved for validation. This procedure was repeated seven times, resulting in seven fold-specific models.

The proposed Transformer architecture was trained using this protocol. The original training configuration was retained, with the exception of the batch size, which was reduced from 2 to 1 to accommodate GPU memory constraints. No additional fine-tuning or post-processing was performed.

E. Evaluation Metric

Performance is evaluated using mean Average Precision (mAP), computed with the official SoccerNet evaluation implementation for the Ball Action Spotting Challenge 2023. Following the evaluation procedure adopted by Baikulov [6], predictions for the publicly available test games are obtained using the corresponding out-of-fold models. The official challenge test set is not publicly available for independent evaluation; therefore, the results reported in this work correspond to the publicly available test data and should not be interpreted as official challenge results.

rest This section explains what has already been done and establishes where your work fits.

I recommend four subsections. 3.1 Soccer Action Spotting Introduce the broader problem of soccer action spotting.

Discuss: - SoccerNet and its datasets. - Temporal localization of soccer events. - The use of deep learning for action spotting. - Relevant approaches to temporal event detection. The objective is to give readers enough background to understand ball action spotting.

You don't need to describe every SoccerNet paper. Focus on the approaches most relevant to your architecture.

F. Ball Action Spotting

This is where you discuss the two references we examined. Baikulov (2023)

Explain: - The slow-fusion 2D/3D architecture. - EfficientNetV2-B0. - The 3D inverted residual blocks. - Multi-stage training. - Transfer learning and fine-tuning with longer sequences. Explain why this is particularly relevant: it provides the architectural reference for your baseline. Solution Key points:
Efficient model architecture for extracting information from video data Multi-stage training (transfer learning, fine-tuning with long sequences) Fast video loading for training (GPU based, no need for preprocessing with extracting images) Model The model architecture employed in this solution utilizes a slow fusion approach, incorporating 2D convolutions in the early stages and 3D convolutions in the later stages. This architectural choice made one of the most significant contributions to the overall metric outcome. It effectively improved the mAP@1 metric on both the test and challenge sets, resulting in an approximate increase of 15% (from 65% to 80%) compared to an approach utilizing 2D CNN early

fusion.

model

The model consumes sequences of grayscale frames. Neighboring frames are stacking as channels for input to the 2D convolutional encoder. For example, if fifteen frames are stacked in sets of three, the outcome would be five input tensors, each consisting of three channels. The shared 2D encoder independently processes these input tensors, producing visual features. The following 3D encoder processes visual features, producing temporal features. Concated temporal features pass through global pooling to compress the spatial dimensions. Then, a linear classifier predicts the presence of actions in the middle frame of the original input sequence.

I choose the following model hyperparameters as a result of the experiments:

Stack threes from 15 grayscale 1280x736 frames skipping every second frame from the original 25 FPS video (equivalent to 15 neighboring frames at 12.5 FPS, about 1.16 seconds window) EfficientNetV2 B0 as 2D encoder 4 inverted residual 3D blocks as 3D encoder (ported from 2D EfficientNet version) GeM as global pooling Multilabel classification with positive labels applied in a 0.6 seconds window (15 frames at 25 FPS) around the action timestamp from annotation You can find more details in the model implementation and experiment configs.

Training

I made several stages of training to obtain 86.47% mAP@1 on the challenge set (87.03% on the test):

Basic training (config). The 2D encoder is initialized with ImageNet-pre-trained weights; other parts start from scratch. Training on Action Spotting Challenge dataset (config). Same weights initialization as in the first stage. Transfer learning (config). 2D and 3D encoders initialized with weights obtained in the second stage. Out-of-fold predictions from the first stage were used for data sampling (more details later). Fine-tuning with long sequences (config). 2D and 3D encoders initialized with weights obtained in the third stage. 2D encoder weights are frozen. Basic training In this challenge, I employed 7-fold cross-validation to tune the training pipeline more precisely. Each labeled game from the dataset is an individual fold.

In short, the resulting training pipeline hyperparameters are:

- Learning rate warmup for the first 6 epochs from 0 to 3e-4, cosine annealing last 30 epochs to 3e-6
- Batch size 4, one training epoch comprises 6000 samples
- Optimizer AdamW with weight decay 0.01
- Focal loss with gamma 1.2
- Model EMA with decay 0.999
- Initial weights for 2D encoder ImageNet pretrained
- Architecture hyperparameters listed above in the model part

Worth writing about sampling techniques during training, which significantly impacts its results. For basic training, a simple but well-working sampling algorithm was used. For each training sample, randomly take a video index from a uniform distribution. Then randomly choose a frame index with an equal chance to sample frames near to actions and remaining frames. Frame near to action if allocated in a 0.36 seconds window (9 frames at 25 FPS) around the action. I tried different ratios, but an equal chance to show empty and event frame worked best. I will introduce a more advanced sampling scheme in the section on transfer learning.

I applied the usual augmentations like horizontal flip, rotation, random resized crop, brightness, motion blur, etc. Of the interesting, there are only two temporal augmentations:

Simulating camera movement (change translation, scale, and angle over time) Randomly shake the frames in sequences (applied 40% chance to change the frame index on +- 1) The models from this training have 79.06% on CV (cross-validation) and 84.26% mAP@1 on the test set (the metric on test split was calculated by the out-of-fold predictions for two folds which include test games). I did not evaluate these models for the challenge set. **Training on Action Spotting Challenge dataset** I built a similar pipeline for videos and classes from Action Spotting Challenge to get good initial weights for the next experiment. Briefly, here are the changes from the previous:

- 377 games in training, 18 in validation split
- 15 classes (all cards-related class was merged because the model consumes grayscale frames)
- 4 warmup epochs and 20 training epochs, one epoch is 36000 samples
- Weight the frame sampling by the effective number of class samples
- Transfer learning

This training uses the results of the previous two. The second one gives excellent initial weights for 2D and 3D encoders. It provides a significant boost (2% mAP@1 on test and CV). That is understandable because the same models were trained on many games with similar input frames and some similar actions.

Basic training gives out-of-fold predictions that I use for sampling in the following way: **sampling**

Take the element-wise maximum between the sampling distribution (introduced above) and predictions, then normalize again to equal probability sums between empty and action frames. The intuition is that there are some hard negative examples in the dataset. Due to many negative samples, such hard examples are rarely sampled during training. We can make something like hard negative mining/sampling with the technique.

Other minor changes compared to basic training:

7 warmup epochs and 35 training epochs Focal loss with gamma 1.2 and alpha 0.4 Models achieve 81.04% on CV, 86.51% on the test, and 86.35% mAP@1 on the challenge set.

Fine-tuning with long sequences Before, I trained all models on relatively short clips (15 frames in 12.5 FPS, 1.16 seconds) to fits the VRAM of a single GPU and to decrease training time. Models obtained good 2D and 3D encoder weights in the previous experiment. So I tried to freeze 2D

weights and fine-tune 3D weights on long sequences to provide more context.

Changes compared to transfer learning experiment:

33 frames in 12.5 FPS (2.6 seconds) LR warmup first 2 epochs from 0 to 1e-3, cosine annealing last 7 epochs to 5e-05 SGD with Nesterov momentum 0.9 Models scored 80.49% on CV, 87.04% on the test, and 86.47% mAP@1 on the challenge set. The score is lower on cross-validation, but it's my best submission on the test and the challenge set.

Prediction and postprocessing Models predict each possible sequence of frames from the videos. Additionally, I make test time augmentation with the horizontal flip. On the challenge set, I used the arithmetic mean of predictions from all fold models.

Postprocessing is very simple. I just used a combination of Gaussian filter and peak detection from SciPy with the following parameters: standard deviation for Gaussian kernel 3.0, peak detection minimal height 0.2, and minimal distance between neighboring peaks 15 frames.

Training and prediction accelerations I optimized the training pipeline to iterate experiments faster and to test more hypotheses.

- Custom multiprocessing video loader with simultaneous use VideoProcessingFramework (GPU decoding) and OpenCV (CPU decoding) workers to optimize hardware utilization
- FP16 with Automatic Mixed Precision
- torch.compile using TorchDynamo backend
- Augmentation on the GPU with kornia

These accelerations allow running epoch (train + val) of basic training in 7 minutes and 10 seconds on a single RTX 3090 Ti. It is impressive because one epoch is 6000 training and approximately 2600 validation examples, each of which is 15 frames in 1280x736 resolution. Also, using source videos without the preprocessing with extracting images allows using any video frame during training and saves disk space.

I applied caching strategy to speed up inference time using the architecture structure. If one saves the last visual features, it is enough to predict with the 2D encoder only one stack of frames when receiving a new one. The 2D encoder is the most time expensive part of the model. Predicting 3D features takes a short time. So this strategy dramatically boosts prediction speed by several times.

My solution is very inspired by the top solutions of the DFL - Bundesliga Data Shootout competition: Team Hydrogen (link) K_mat (link) Camaro (link) ohkawa3 (link)

G. Wang et al. (2023)

Authors summary

based on J. Hong et al. [5] E2E-Spot Here we talk about the solution with the paper.

Our method, Boosted Model Ensembling (BME), is based on the end-to-end baseline model, E2E-Spot, as presented in [10]? citation.

We generate several variants of the E2E-Spot model to create a candidate model set and propose a strategy for selecting appropriate model members from this set while assigning appropriate weights to each selected model. More details can be found in [35]? citation.

The resulting ensemble model takes into account uncertainties in event length, optimal network architectures, and optimizers, making it more robust than the baseline model. Our approach has the potential to handle various video event analysis tasks. Explain: - Boosted Model Ensembling. - Candidate model generation. - Model selection and ensemble weighting. - The importance of model diversity. You don't need to describe every implementation detail. The important point is that it represents another successful approach to the same challenge. 3.3 Convolutional Temporal Modeling Explain the principles behind 3D CNNs.

Discuss: - 2D convolution extracts spatial features. - 3D convolution operates over spatial and temporal dimensions. - Temporal convolutions aggregate information from neighboring frames. - Slow fusion allows spatial feature extraction before temporal processing. Then explain the advantages of this approach, such as local temporal processing and efficient hierarchical feature extraction.

This subsection prepares the reader for your baseline architecture.

H. Transformer-Based Temporal Modeling

Introduce Transformers and self-attention.

Explain: - Temporal tokens. - Query, key and value representations. - Self-attention. - Positional embeddings. - Temporal aggregation. Discuss relevant applications of Transformers in video understanding and temporal action localization.

Finally, explain that your work investigates their use in ball action spotting, specifically as an alternative to the 3D CNN temporal encoder.

Important: Your actual Transformer architecture and its hyperparameters should not be described here. Those belong in Proposed Method.

askldjkljekle

I. Transformer-Based Video Modeling

Introduce self-attention, positional information, and the use of Transformer encoders to model temporal sequences. Include work on temporal action spotting and video modeling that is technically relevant to your architecture. For instance, Transformer-based temporal reasoning has already been explored in soccer action spotting, so your paper should distinguish your particular replacement and evaluation from prior work rather than claim to introduce Transformers to the task.

J. C. Transformer-Based Temporal Modeling

To investigate the use of self-attention for temporal feature modeling, we replace the original 3D convolutional encoder with a Transformer encoder while retaining the shared 2D visual feature extractor. The input video is divided into groups of three consecutive grayscale frames, and each group is

processed independently by the 2D encoder. The resulting feature maps are projected to a common feature dimension of 192 channels.

Unlike the original architecture, which performs temporal modeling on spatial feature maps, the proposed approach first applies adaptive average pooling to each feature map, producing a sequence of five 192-dimensional feature vectors for a 15-frame input. Learned positional embeddings are added to the sequence to provide information about the temporal ordering of the feature vectors.

The resulting sequence is processed by a Transformer encoder consisting of two layers with eight attention heads and an embedding dimension of 192. Self-attention allows each temporal token to incorporate information from the other tokens in the input sequence. The encoded representations are then aggregated by temporal average pooling to obtain a single 192-dimensional video representation.

Finally, a linear classification layer maps this representation to the output action classes. This architecture replaces the original 3D convolutional temporal encoder and its subsequent feature projection and concatenation with a Transformer-based temporal encoder and average-pooling operation. The effect of this architectural change is evaluated using the same action spotting task and evaluation metric as the baseline.

K. Convolutional Temporal Modeling

L. Transformer-Based Video Modeling

V. Proposed Method

This is probably the most important section of your paper.

The reader should understand exactly what you implemented and how the two architectures differ.

I recommend five subsections.

A. Task Definition

Mathematically define the problem.

Explain: - Input: a sequence of video frames. - Output: probabilities for the ball action classes. - Labels: actions associated with timestamps. - Prediction: whether an action occurs at the center timestamp of the input sequence. You should explicitly explain your label construction.

For example, if you use a 0.6-second positive labeling window, explain how frames within this interval are assigned positive labels.

You can introduce notation such as:

$$X = \{I_1, I_2, \ldots, I_T\}$$

where I_t represents a video frame.

The model predicts:

$$\hat{y} = f(X; \theta)$$

where $\hat{y}$ contains the predicted action probabilities. Define the input video sequence, the action classes, the timestamp-based labels, and the prediction target. Explain whether the model predicts the action at the center frame or predicts a temporal sequence of action probabilities.

Your draft describes labels expanded around the annotated timestamp. Make that explicit, including the window size, because it affects what the model is learning.

This novel task of ball action spotting consists in localizing the exact time when two types of actions related to the soccer ball occur: pass and drive. These actions are anchored with a single timestamp corresponding to the exact time the ball leaves a player's foot for a pass event and when a player touches the ball to control it for a drive event.
Ball action spotting involves identifying both the timing and type of ball-related actions across 12 classes in our dataset. Each action is marked by a single timestamp. In contrast to the SoccerNet Action Spotting Challenge, our videos feature a higher density of actions.

B. Soccer Action Spotting

ball action spotting, focusing on retrieving all timestamps related to the soccer ball change of state,

C. Ball Action Spotting

The SoccerNet Ball Action Spotting Challenge was introduced to address the problem of temporally localizing ball-related actions in broadcast soccer videos. Unlike general action spotting, ball-related events such as passes and drives are often densely distributed and require precise temporal localization.

Among the solutions proposed for the 2023 challenge, Baikulov achieved first place with an efficient deep learning architecture combining two-dimensional and three-dimensional convolutional operations [6]. The proposed architecture follows a slow-fusion strategy, in which a shared 2D convolutional encoder independently extracts visual features from groups of consecutive grayscale frames. These features are subsequently processed by a 3D convolutional encoder to capture temporal information. The resulting representations are aggregated through generalized mean pooling and used for multilabel action classification.

In addition to the model architecture, the solution introduced a multi-stage training strategy involving pre-training on the SoccerNet Action Spotting dataset, transfer learning, and fine-tuning with longer video sequences. The use of out-of-fold predictions for sample selection and an efficient video loading pipeline further contributed to the final performance. The solution achieved 86.47% mAP@1 on the challenge set [6].

Another notable approach was proposed by Wang et al., who obtained second place in the 2023 challenge with Boosted Model Ensembling (BME) [7]. Their method is based on the E2E-Spot baseline and focuses on improving performance through the generation and combination of multiple candidate models. These models differ in their training sample configurations, feature extraction architectures, and optimization strategies. The candidate architectures include RegNet and EfficientNet backbones incorporating Gate-Shift Modules, while

the training strategies consider AdamW and stochastic weight averaging.

Rather than relying on a conventional fixed ensemble, BME iteratively selects candidate models and determines their contribution according to improvements in validation mAP@1. The resulting ensemble achieved 86.37% mAP@1 on the challenge test set [7].

These approaches demonstrate the importance of both spatiotemporal feature extraction and training strategies for accurately spotting ball-related events. In particular, the slow-fusion architecture proposed by Baikulov provides the convolutional baseline considered in this work. We investigate an alternative approach to temporal feature modeling by replacing its 3D convolutional encoder with a Transformer encoder, while retaining the shared 2D visual feature extractor.

D. Baseline Architecture

Describe Baikulov's architecture as the reference architecture you reproduce or adapt.

Include a diagram.

Explain the processing pipeline: 1. Sample frames from the video. 2. Group neighboring grayscale frames into stacks of three. 3. Apply the shared 2D EfficientNetV2-B0 encoder. 4. Process the resulting feature maps using the 3D temporal encoder. 5. Aggregate features using pooling. 6. Predict action probabilities using a classification layer. Include tensor dimensions wherever possible.

For example, assuming five stacks and a 192-channel projected representation, describe the actual dimensions produced by your implementation rather than relying on this example.

E. Transformer-Based Temporal Encoder

This is your main architectural contribution.

Explain the architecture step by step.

Based on your previous draft: 1. Divide the 15-frame input into five groups of three frames. 2. Process each group using the shared 2D encoder. 3. Project the extracted features to 192 dimensions. 4. Apply adaptive average pooling to obtain five temporal feature vectors. 5. Add learned positional embeddings. 6. Process the sequence with a Transformer encoder. 7. Apply temporal average pooling. 8. Use a linear classifier to predict the action classes. Include the actual Transformer configuration: Parameter Value Embedding dimension 192 Number of layers 2 Attention heads 8 Number of temporal tokens 5 Positional encoding Learned Temporal aggregation Average pooling

Also report the feed-forward dimension, dropout, normalization, and activation function from your implementation.

Explain why you chose these values, if you have experimental justification.

F. Architectural Differences

I strongly recommend adding this subsection.

A table will make your contribution much easier to understand. Component CNN baseline Transformer 2D backbone EfficientNetV2-B0 Same Input frames 15 15 Frame grouping 3 3 Temporal encoder 3D CNN Transformer Spatial aggregation As implemented in baseline Adaptive average pooling Temporal aggregation Baseline pooling Average pooling Classifier Linear Linear

Verify every entry against your actual code.

In particular, explain that spatial pooling before attention changes the feature representation as well as the temporal modeling operation. This distinction is important for interpreting the comparison.

G. Training and Inference

Describe the training and prediction pipelines.

Include: - Input resolution. - Frame sampling rate. - Number of input frames. - Label construction. - Loss function. - Optimizer. - Learning rate and scheduler. - Batch size. - Data augmentation. - Initialization and pretrained weights. - Number of epochs. - Prediction procedure. - Postprocessing. For inference, explain how frame-wise probabilities are converted into timestamp predictions.

For example, describe Gaussian smoothing and peak detection if you retain those operations.

H. Baseline Architecture

Research question is supposed to be about replacing the temporal 3D CNN with a Transformer Describe the existing pipeline with a diagram and a concise explanation: 1. Sample frames from the video. 2. Stack neighboring grayscale frames into groups of three. 3. Extract visual features using the shared 2D encoder. 4. Process the feature sequence using the 3D temporal encoder. 5. Pool the features and predict action probabilities. The draft already contains most of this information. Add tensor dimensions at each major stage, where possible, so that the reader can reproduce the architecture.

I. Baseline Architecture

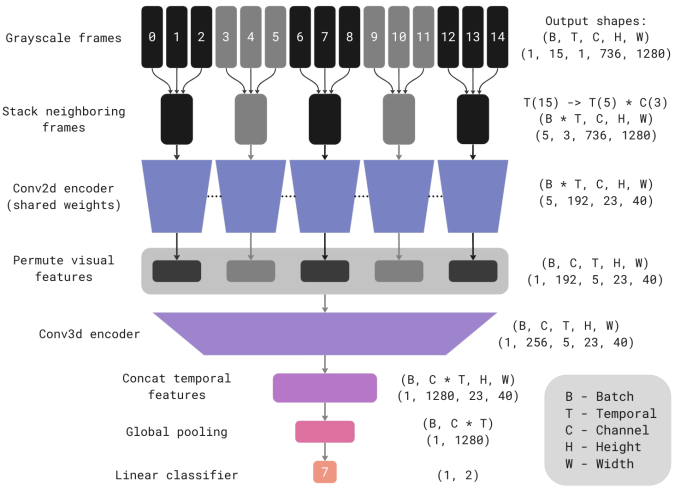


Fig. 1. Model second solution

This should be the main contribution. Specify: - Which features enter the Transformer: raw frame embeddings, spatially pooled 2D features, or intermediate CNN feature maps.

- The projection from visual feature dimension to Transformer embedding dimension. - The positional encoding method. - Number of Transformer layers and attention heads. - Feed-forward dimension, normalization, dropout, and activation. - How the temporal representation is pooled or selected for classification. Explain which parts remain unchanged from the baseline and which are replaced.

J. Training and Inference

Describe the common training setup, loss, label expansion, augmentation, and prediction postprocessing. If the Transformer requires different optimization settings or a different sequence length, report those differences clearly.

A key methodological choice is whether you train the Transformer from scratch or initialize it from pretrained weights. That choice affects how you interpret the results, particularly with limited labeled data.

K. Temporal Feature Representation

L. Transformer Encoder

M. Prediction and Postprocessing

VI. Experimental Setup

The purpose of this section is reproducibility.

Another researcher should be able to understand how you performed the experiments.

A. Dataset

Describe the actual challenge dataset you used.

Include: - Dataset source. - Number of matches. - Number of annotated events. - Number of action classes. - Video resolution and frame rate. - Training, validation and test splits. - Challenge evaluation set. Be especially careful about the challenge year: your previous draft mixed 2023 and 2024 dataset descriptions.

Use the correct dataset statistics for your experiment. The dataset is provided by Footovision and is composed of 7 games with a total of 11,041 annotated timestamps, as well as 2 extra segregated games for the challenge set.

The dataset is [8]

The data consists of 7 videos of English Football League games, each broadcast in 720p quality. We don't provide extracted features to encourage the development of end-to-end methods. The challenge set is composed of 2 separate games. Additionally, participants are encouraged to leverage the 500 unannotated games from the Action Spotting challenge as a resource for this task.

The dataset available is relatively small compared to the action spotting task, encouraging participants to explore semi-supervised, selfsupervised, or transfer learning methods using the 500 unlabeled broadcast games.

A significant enhancement this year is the inclusion of 12 classes of ball actions, requiring more precise classification. Consequently, simple categorizations like "pass" are insufficient as specific pass types must be clearly identified.

We provide annotation for all two types of soccer ball actions: Pass, Drive. In 2024, we extend it to 12 different classes of soccer ball actions: Pass, Drive, Header, High Pass, Out, Cross, Throw In, Shot, Ball Player Block, Player Successful Tackle, Free Kick, Goal

B. Implementation Details

Describe your technical environment.

For example: - PyTorch version. - GPU hardware. - CUDA version. - Input resolution. - Batch size. - Optimizer. - Training duration. - Mixed precision. - Random seeds. Also report the relevant model hyperparameters.

C. Evaluation Metrics

We measure Average Precision within a tolerance of δ frames (AP @ δ). AP is computed for each event class, and mAP is the mean across classes. We focus on tight tolerances such as $\delta = 1$ and $\delta = 2$. Precise temporal events are rare as a percentage of frames (0.2–2.9%), so metrics such as frame-level accuracy are not meaningful for precise spotting Explain mAP@1.

Describe: - Average Precision. - Mean Average Precision across action classes. - The one-second temporal tolerance. - The official evaluation protocol. If you measure inference speed or memory, explain those metrics here too. Unlike the SoccerNet Action Spotting Challenge, that uses the tight average-mAP ([1-5] seconds tolerance), this challenge focuses on accurately spotting the ball events with the mAP@1 (1 second tolerance).

D. Experimental Configurations

Clearly identify your experiments.

I suggest the following table: Experiment Temporal encoder Purpose E1 3D CNN Baseline E2 Transformer Proposed architecture E3 Transformer, alternative configuration Ablation, if available

Try to keep the experiments controlled.

Ideally, the baseline and Transformer should use the same: - Dataset split. - Input frames. - 2D backbone. - Training data. - Label construction. - Evaluation procedure. - Postprocessing. If you cannot keep everything identical, document the differences.

The goal here is to make the experiment reproducible and the comparison defensible. Subsection What it should contain Dataset Number of matches, classes, annotations, and data split Implementation Framework, hardware, input resolution, model settings Evaluation metrics mAP@1 and any efficiency metrics Experimental configurations Baseline, Transformer variant, and ablations Training protocol Initialization, optimizer, schedule, seeds, and validation procedure

Keep the primary comparison as controlled as possible: use the same data splits, input frames, 2D feature extractor, label construction, postprocessing, and evaluation implementation for both models.

If you change several things at once — for example, the temporal encoder, the input length, and the loss — then a performance difference cannot be attributed confidently to the Transformer.

E. Implementation Details

F. Evaluation Metrics

G. Experimental Configurations

VII. Results and Discussion

This section should answer your research question, not just present numbers.

I recommend five subsections.

A. Main Comparison

This isn't true, TODO: rewrite and check
We use the 3D CNN-based architecture from the 2023 challenge as the reference for our design. Since the proposed model was trained on the 2024 data, results from the two challenge years are not a controlled comparison of temporal architectures. Differences in dataset composition, training data, and evaluation conditions must be considered when interpreting the results.

This is your most important results table.

For example: Architecture mAP@1 Parameters Inference time 3D CNN baseline TBD TBD TBD Transformer TBD TBD TBD

Use actual measurements, and clarify whether inference time is measured per frame, sequence, or video.

Discuss: - Which architecture performs better? - What is the absolute difference in mAP? - Are the differences consistent across classes or folds? - Does the Transformer introduce computational overhead? Don't interpret a small difference as significant without suitable repeated experiments.

B. Effect of Temporal Context

Investigate how the length of the input sequence affects performance.

For example: - 15 frames. - 33 frames. - Other sequence lengths, if you tested them. Explain whether longer temporal context improves spotting performance.

If you compare sequence lengths, make sure the sampling rate and temporal coverage are clearly specified.

C. Ablation Study

Investigate individual architectural choices.

Possible experiments: - Transformer depth. - Number of attention heads. - Embedding dimension. - Positional embeddings. - Temporal pooling strategy. You don't need to test all of these. Select the experiments you have actually performed. 6.4 Error Analysis This section is often overlooked but can make your paper considerably stronger.

Analyze: - False positives. - Missed events. - Closely spaced actions. - Confusion between action classes. - Temporal localization errors. Include qualitative examples if possible.

For instance, show a short video timeline with ground-truth timestamps and predicted timestamps.

D. Limitations

Be honest about the limitations.

Potential points: - Limited size of the annotated dataset. - Small number of temporal tokens. - Spatial information lost through global pooling before attention. - Computational requirements. - Lack of extensive hyperparameter tuning. - Differences between the original winning solution and your reproduction. Only include limitations that genuinely apply. Your current proposed subsections are sensible, but each should answer a specific question.

E. Main Comparison

Compare the CNN and Transformer using the same evaluation protocol. Report mAP@1, absolute difference in percentage points, and, if feasible, variation across random seeds or folds.

F. Effect of Temporal Context

Test whether performance changes when the input sequence becomes longer. To isolate the effect of the encoder, use comparable temporal windows for both architectures.

G. Ablation Study

Test one design choice at a time, such as embedding dimension, number of Transformer layers, or positional encoding. Choose only a few ablations that help explain the main result.

H. Error Analysis

Investigate false positives, missed actions, closely spaced events, and differences between action classes. Use examples or per-class metrics where possible.

I'd also report inference latency and peak GPU memory if efficiency is part of the research question. Parameter count alone is not a sufficient measure of computational efficiency.

One distinction matters for your current draft: the leaderboard table is useful context, but it is not a substitute for a controlled baseline comparison. Your model must be evaluated on the same split and with the same protocol before you can attribute a change to the temporal encoder.

We evaluate the proposed Transformer-based model on the 2024 ball-action spotting challenge using the official evaluation protocol. We report the primary challenge metric and describe the evaluation split and inference procedure.

Here we need to compare to other results.
After the table

I. Effect of Temporal Context

J. Ablation Study

K. Error Analysis

We examine the model's predictions to identify common false positives, missed actions, and temporal localization errors. Where ground-truth labels are available, we also analyze performance by action class.

TABLE I

BALL ACTION SPOTTING LEADERBOARD. THE MAIN METRIC FOR THE LEADERBOARD AND BEST PERFORMANCES ARE IN BOLD. TEAM NAMES WITH A SUPERSCRIPT HAVE PROVIDED A SUMMARY THAT MAY BE FOUND IN APPENDIX **??** OR IN SECTION **??** FOR THE WINNING TEAM.

Participants	mAP @1	mAP @2	mAP @3	mAP @4	mAP @5	Average-mAP tight
Ruslan Baikulov^{B1}	**86.47**	**87.98**	**88.28**	**88.18**	**87.95**	**87.91**
FDL@ZLab^{B2}	83.39	85.19	85.81	86.00	86.19	85.45
BASIK^{B3}	82.06	83.39	83.86	84.04	83.91	83.57
FC Pixel Nets^{B4}	81.89	83.22	83.97	83.85	84.02	83.50
play	79.74	82.58	84.06	84.49	84.34	83.29
Baseline [**?**]*	62.72	69.24	72.57	74.29	74.80	71.21

TABLE II

BALL ACTION SPOTTING LEADERBOARD. THE MAIN METRIC FOR THE LEADERBOARD AND BEST PERFORMANCES ARE IN BOLD.

L. Limitation

The current comparison does not isolate the effect of replacing 3D convolutions with self-attention. The proposed model also changes spatial and temporal feature aggregation. A controlled comparison using both architectures trained and evaluated under the same conditions would be needed to attribute performance differences more specifically to the temporal modeling approach

VIII. CONCLUSION AND FUTURE WORK

A. Conclusion and Future Work

Keep this section relatively short.

Structure it in three paragraphs.

Paragraph 1 — Objective and method

Summarize the problem and your proposed architecture.

Paragraph 2 — Main findings

Report the actual experimental results and explain what they demonstrate about CNN-based and Transformer-based temporal modeling.

Paragraph 3 — Future work

Possible directions: - Spatial-temporal attention instead of spatially pooled temporal tokens. - More efficient Transformer architectures. - Longer temporal sequences. - Pretraining on larger soccer video datasets. - Combining convolutional and attention-based temporal representations. Avoid introducing new experimental claims in the conclusion.

ACKNOWLEDGMENT

Acknowledgment

REFERENCES

[1] A. Deliège, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasrollahi, B. Ghanem, T. B. Moeslund, and M. V. Droogenbroeck, "Soccernet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos," 2021. [Online]. Available: https://arxiv.org/abs/2011.13367

[2] Y. Li, L. Chen, R. He, Z. Wang, G. Wu, and L. Wang, "Multisports: A multi-person video dataset of spatio-temporally localized sports actions," 2021. [Online]. Available: https://arxiv.org/abs/2105.07404

[3] S. Giancola, M. Amine, T. Dghaily, and B. Ghanem, "Soccernet: A scalable dataset for action spotting in soccer videos," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, June 2018, p. 1792–179210. [Online]. Available: http://dx.doi.org/10.1109/CVPRW.2018.00223

[4] A. Cioppa, S. Giancola, V. Somers, F. Magera, X. Zhou, H. Mkhallati, A. Deliège, J. Held, C. Hinojosa, A. M. Mansourian, P. Miralles, O. Barnich, C. De Vleeschouwer, A. Alahi, B. Ghanem, M. Van Droogenbroeck, A. Kamal, A. Maglo, A. Clapés, A. Abdelaziz, A. Xarles, A. Orcesi, A. Scott, B. Liu, B. Lim, C. Chen, F. Deuser, F. Yan, F. Yu, G. Shitrit, G. Wang, G. Choi, H. Kim, H. Guo, H. Fahrudin, H. Koguchi, H. Ardö, I. Salah, I. Yerushalmy, I. Muhammad, I. Uchida, I. Be'ery, J. Rabarisoa, J. Lee, J. Fu, J. Yin, J. Xu, J. Nang, J. Denize, J. Li, J. Zhang, J. Kim, K. Synowiec, K. Kobayashi, K. Zhang, K. Habel, K. Nakajima, L. Jiao, L. Ma, L. Wang, L. Wang, M. Li, M. Zhou, M. Nasr, M. Abdelwahed, M. Liashuha, N. Falaleev, N. Oswald, Q. Jia, Q.-C. Pham, R. Song, R. Hérault, R. Peng, R. Chen, R. Liu, R. Baikulov, R. Fukushima, S. Escalera, S. Lee, S. Chen, S. Ding, T. Someya, T. B. Moeslund, T. Li, W. Shen, W. Zhang, W. Li, W. Dai, W. Luo, W. Zhao, W. Zhang, X. Yang, Y. Ma, Y. Joo, Y. Zeng, Y. Gan, Y. Zhu, Y. Zhong, Z. Ruan, Z. Li, Z. Huang, and Z. Meng, "Soccernet 2023 challenges results," *Sports Engineering*, vol. 27, no. 2, 2024. [Online]. Available: http://dx.doi.org/10.1007/s12283-024-00466-4

[5] J. Hong, H. Zhang, M. Gharbi, M. Fisher, and K. Fatahalian, "Spotting temporally precise, fine-grained events in video," 2022. [Online]. Available: https://arxiv.org/abs/2207.10213

[6] R. Baikulov, "Solution for soccernet ball action spotting challenge 2023," https://github.com/lRomul/ball-action-spotting, 2023, first-place challenge solution.

[7] L. Wang, H. Guo, and B. Liu, "A boosted model ensembling approach to ball action spotting in videos: The runner-up solution to cvpr'23 soccernet challenge," 2023. [Online]. Available: https://arxiv.org/abs/2306.05772

[8] A. Deliège, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasrollahi, B. Ghanem, T. B. Moeslund, and M. V. Droogenbroeck, "Soccernet-v2 : A dataset and benchmarks for holistic understanding of broadcast soccer videos," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2021.